

SOFIAA OS

N.E.X.O. — Nexus Extension de Conocimiento y Operaciones

Reporte Doctoral de Arquitectura e Implementación

Experience Graph · Decay Engine · Semantic Retrieval · Reflection Engine

SOFIAA OS · Sistema de Inteligencia Personal · Sprints N-0 → M-4B

Versión 2.0 · Julio 2026

1. Resumen Ejecutivo

N.E.X.O. (Nexus Extension de Conocimiento y Operaciones) es la capa de memoria a largo plazo de SOFIAA OS. Es el sistema que transforma lo que el usuario ve, lee y procesa en conocimiento estructurado, persistente y cognitivamente relevante. A diferencia de un historial de chat que se desvanece, N.E.X.O. construye un grafo de experiencia semántica que crece con el usuario, aprende sus patrones de interés y decae orgánicamente cuando la información deja de ser relevante.

El sistema opera sobre un principio inspirado en la neurociencia cognitiva: la memoria no es almacenamiento infinito, sino un proceso activo de consolidación, refuerzo y olvido selectivo. Un nodo nuevo entra con peso 0.60, se refuerza (+0.03) cada vez que SOFIAA lo referencia en conversación, y decae exponencialmente ($\lambda = \ln(2)/t_{1/2}$) en función de la categoría de contenido. Los nodos que caen por debajo de 0.05 son eliminados permanentemente — el sistema olvida, como lo hace la memoria humana.

N.E.X.O. tiene siete motores especializados: Ingest Pipeline (clasificación LLM), Decay Engine (olvido adaptativo), Reflection Engine (generación de insights), Attention Engine (refuerzo por uso), Semantic Retrieval (ranking híbrido coseno+peso), PDF Library (ingesta de documentos), y Context Injection (inyección en el system prompt). Juntos forman la capa cognitiva más fundamental de SOFIAA.

Motor	Sprint	Función Central
Ingest Pipeline (LLM Classifier)	N-2	Clasifica contenido web → NexoNode estructurado via Groq
Decay Engine	N-3	Decaimiento exponencial $\lambda = \ln(2)/t_{1/2} \cdot \text{CRON 3 AM diario}$
Context Injection	N-4	Inyecta top-N nodos en el system prompt de SOFIAA
Reflection Engine	M-0	Genera insights cognitivos desde patrones del grafo
PDF Library	M-1	Procesa PDFs → múltiples NexoNodes por documento
Attention Engine	M-2	Refuerza nodos usados en conversación via waitUntil()
Semantic Retrieval	M-4	Ranking híbrido: $\text{coseno}(\text{query}, \text{node}) * 0.65 + \text{peso} * 0.35$

2. Schema de Datos — NexoNode

El NexoNode es la unidad fundamental de N.E.X.O. Cada nodo representa un fragmento de conocimiento capturado del usuario: una página web, un restaurante, un producto, un artículo de investigación o un insight generado automáticamente. La interfaz está diseñada para ser expresiva pero económica — cada campo tiene un propósito cognitivo claro.

2.1 Campos Core del NexoNode

Campo	Tipo	Descripción Cognitiva
id	string	'nexo:{category}:{slug}' — identificador único semántico
type	'captured' 'insight'	Capturado por usuario vs generado por Reflection Engine
category	NexoCategory	Categoría semántica (food, work, travel, research, etc.)
title	string	Título legible para el LLM — se usa en el prompt
summary	string	Resumen ≤200 chars extraído por el LLM Classifier
entities	NexoEntities	Entidades clave: place, person, product, price, brand, hashtags
weight	number 0-1	Relevancia actual — decae exponencialmente con el tiempo
importanceScore	number 0-1	Importancia asignada en ingesta (0=spam, 1=muy relevante)
decayRate	number	Tasa individual de decaimiento. Default: 0.05 web, 0.02 PDFs, 0.02 insights
lastReinforced	number (ms)	Último refuerzo — resetea el reloj del decay
reinforceCount	number?	Veces seleccionado por el Attention Engine (Sprint M-2)
embedding	number[]?	Vector semántico para similitud coseno (Sprint M-4)

2.2 NexoCategory — Las 9 Categorías Semánticas

El clasificador LLM asigna una de 9 categorías a cada nodo. La categoría determina la tasa de decaimiento (NEXO_DECAY_DAYS): las memorias de trabajo y marca son más estables; comida, social y media decaen más rápido, reflejando su naturaleza más efímera.

Categoría	Semántica	Half-Life (días)	Ejemplo de Captura
brand_identity	Brand DNA, identidad de marca (PROMETEO)	90	Brief de cliente, arquetipo de marca
work	Artículos, proyectos, herramientas profesionales	30	Tutorial de Next.js, propuesta de proyecto
research	Papers, noticias, investigación académica	30	Estudio de mercado, artículo IEEE
travel	Lugares, hoteles, itinerarios	21	Hotel en CDMX, aeropuerto Monterrey

Categoría	Semántica	Half-Life (días)	Ejemplo de Captura
food	Restaurantes, recetas, platillos	14	Sushi Zona Rosa, Tacos El Franc
shopping	Productos, precios, comparaciones	14	iPhone 16 Pro en Liverpool, Sony A7
other	Contenido sin categoría específica	14	Nodo fallback del clasificador
social	Posts de redes sociales, perfiles	10	Thread de Twitter, post de LinkedIn
media	Videos, podcasts, entretenimiento	10	Podcast sobre IA, serie en Netflix

2.3 NexoSource — Canales de Ingesta

Source	Canal	Descripción
chrome_extension	Sprint N-1	Extensión de Chrome — captura activa mientras navega
pwa_share	Sprint N-6	PWA Share Target — compartir desde móvil
screenshot	Sprint N-2	Imagen/captura de pantalla procesada con Gemini Vision
manual	Core	Pegado manual en el chat de SOFIAA
pdf_library	Sprint M-1	Documento PDF subido a la Biblioteca Personal
tec_bii	Sprint T2-1	Entidades TEC Bii publicadas al grafo (Cognitive Publisher)
prometeo	Sprint P-1	Brand DNA y entidades PROMETEO (6 nodos por cliente)
marketing_sofia	Sprint P-8	Clientes y métricas SMM publicadas al grafo

3. Constantes del Sistema — Parámetros Cognitivos

N.E.X.O. está gobernado por un conjunto de constantes que definen el comportamiento cognitivo del sistema. Estas constantes son el resultado de un proceso de calibración que equilibra la retención de información útil con el olvido de información irrelevante, y la relevancia semántica con la frecuencia de uso.

Constante	Valor	Propósito
NEXO_INITIAL_WEIGHT	0.60	Peso inicial de todo nodo nuevo — 'relevante por defecto'
NEXO_DECAY_RATE	0.05	Tasa de decaimiento por defecto para capturas web
NEXO_PRUNE_THRESHOLD	0.05	Peso mínimo antes de eliminación definitiva (olvido)
NEXO_MAX_CONTEXT_NODES	5	Nodos máximos inyectados en el system prompt por conversación
NEXO_PROACTIVE_THRESHOLD	0.65	Score mínimo para activar mención proactiva por SOFIAA
NEXO_MAX_PROACTIVE_NODES	2	Máx nodos proactivos por request — evita saturar el prompt
ATTEND_DELTA	0.03	Boost de peso por atención del Attention Engine
Refuerzo manual	0.20	Boost cuando SOFIAA referencia explícitamente un nodo
Boost por upsert (URL dup)	0.15	Boost al capturar una URL ya existente en el grafo
decayRate insights	0.02	Tasa de decay para nodos tipo 'insight' — duran más
decayRate PDFs	0.02	Tasa de decay para nodos de biblioteca PDF
weight PDFs inicial	0.75	Peso inicial mayor para documentos subidos intencionalmente
weight insights inicial	0.85	Peso alto — los insights son conocimiento sintetizado

4. LLM Classifier — Motor de Clasificación (Sprint N-2)

El LLM Classifier es el primer motor que procesa cada captura. Su función es transformar contenido web crudo (URL, título, texto DOM) en un NexoNode estructurado con semántica rica. Opera con Groq (llama-3.3-70b-versatile) en modo JSON estricto, con temperatura 0.1 para máxima consistencia en la clasificación.

4.1 Pipeline de Ingesta

El endpoint POST /api/nexo/ingest orquesta 7 pasos, todos serverless (Next.js API route, runtime nodejs):

1. Auth: `verifyIdToken(Firebase)` → uid
2. Parse + Validar: `title, text, source, capturedAt` requeridos
3. Sanitizar: `title.slice(0,300), text.slice(0,8000)`
4. `classifyNexoPayload(clean)` → { category, importanceScore, summary, entities, slug }
5. `nodeId = 'nexo:{category}:{slug}'`
6. `generateEmbedding(title + '. ' + summary)` → embedding vector (Sprint M-4)
7. `upsertNexoNode(uid, node)` → guarda o refuerza si URL ya existe
8. `logNexoEvent(uid, event)` → registro para N.O.R.A.

4.2 Prompt del Clasificador

El clasificador usa un system prompt de 13 líneas con reglas explícitas de categorización y un prompt de usuario que incluye URL, título, fuente y hasta 3,000 caracteres del texto DOM. La respuesta JSON contiene: category, importanceScore (0.0-1.0), summary (≤200 chars en español), entities (place/person/product/price/brand/hashtags), y slug en kebab-case sin acentos.

4.3 Fallback Heurístico

Si Groq no responde o devuelve JSON inválido, el sistema activa `heuristicClassify()`: un clasificador rule-based con regex en español que detecta restaurantes (restau|comida|taco|sushi), shopping (precio|compra|tienda), viajes (hotel|vuelo|airbnb), investigación (paper|estudio|análisis), trabajo (proyecto|empresa|cliente), social (instagram|twitter|linkedin) y media (youtube|spotify|netflix). Retorna importanceScore=0.5 sin llamar al LLM.

4.4 Deduplicación por URL

`upsertNexoNode()` verifica si ya existe un nodo con la misma URL antes de crear uno nuevo. Si existe, en lugar de duplicar aplica un boost de +0.15 al peso y actualiza lastReinforced. Este mecanismo evita la proliferación

de nodos redundantes cuando el usuario visita la misma página múltiples veces, reforzando en cambio su relevancia.

5. Decay Engine — Olvido Adaptativo (Sprint N-3)

El Decay Engine implementa olvido cognitivo inspirado en la curva de Ebbinghaus. No todos los nodos decaen igual: la categoría del contenido determina su 'media vida' (half-life), reflejando la naturaleza volátil o estable del tipo de información. Un post de redes sociales se vuelve irrelevante en días; el Brand DNA de un cliente puede mantenerse relevante por meses.

5.1 Fórmula de Decaimiento Exponencial

El sistema usa decaimiento exponencial con lambda adaptativo por categoría. La fórmula es matemáticamente equivalente a la curva de retención de Ebbinghaus a nivel individual:

```
λ = ln(2) / halfLifeDays          // Lambda basado en la media vida de la categoría
newWeight = weight × e^(-λ × daysSinceLastReinforced)
```

Donde `daysSinceLastReinforced = (Date.now() - lastReinforced) / 86_400_000`. El campo `lastReinforced` se resetea cada vez que el Attention Engine refuerza el nodo, reiniciando efectivamente el reloj del decay — igual que la repetición espaciada en técnicas de estudio.

5.2 CRON Schedule — Vercel 3 AM CST

El Decay Engine corre como Vercel Cron Job a las 9:00 AM UTC (equivalente a las 3:00 AM CST México). La cadencia diaria garantiza que el sistema actualice pesos cada 24 horas sin acumulación de deuda de decaimiento. El endpoint `GET /api/nexo/decay` usa `collectionGroup('nexo_nodes')` para descubrir automáticamente todos los UUIDs con nodos activos, procesa cada usuario secuencialmente para evitar exceder cuotas de escritura, y registra un `NexoEvent` de tipo 'pruned' por cada eliminación para el panel N.O.R.A.

5.3 Proceso por Usuario

Paso	Operación	Resultado
1. Fetch	<code>col.get()</code> — todos los nodos del usuario	Array de <code>NexoNode</code> con peso actual
2. Calcular	λ adaptativo por category + días transcurridos	<code>newWeight</code> por nodo
3. Decisión	<code>newWeight < NEXO_PRUNE_THRESHOLD (0.05)?</code>	Si sí → eliminar; si no → actualizar
4. Batch	<code>db.batch()</code> con updates + deletes	Escritura atómica a Firestore
5. Log	<code>logNexoEvent</code> si <code>pruned > 0</code>	Registro de olvido en N.O.R.A.

6. Context Injection — Inyección en System Prompt (Sprint N-4)

Context Injection es el mecanismo que conecta el grafo de memoria con las conversaciones de SOFIAA. Antes de cada respuesta del LLM, el sistema recupera los nodos más relevantes del grafo del usuario y los inyecta en el system prompt, dándole a SOFIAA acceso a la memoria personal del usuario sin que este tenga que mencionarla explícitamente.

6.1 getSemanticNexoContext() — Función Principal (Sprint M-4)

La función `getSemanticNexoContext(uid, queryText, limit=5)` implementa el retrieval semántico híbrido. Recibe el texto del último mensaje del usuario como `queryText` y devuelve un `NexoContext` con los 5 nodos más relevantes. El flujo es:

```
1. Fetch todos los nodos activos (weight > 0.05) sin ORDER BY → permite re-ranquear en memoria
2. generateQueryEmbedding(queryText) → vector semántico de la query
3. Para cada nodo: score = hybridScore(queryEmbedding, node.embedding, node.weight)
4. scored.sort() descendente → top-5 seleccionados
5. Proactive nodes: scored donde score ≥ 0.65 (máx 2)
6. count().get() → totalNodes (KPI para el dashboard)
```

6.2 HybridScore — La Fórmula de Relevancia

El score híbrido combina dos dimensiones de relevancia: similitud semántica con la query actual (65% del peso) y relevancia histórica del nodo expresada por su `weight` (35% del peso). Este balance garantiza que los nodos contextuales a la conversación actual tengan prioridad, sin ignorar el historial de uso.

```
hybridScore = cosineSimilarity(query, node) * 0.65 + node.weight * 0.35
```

Si no hay embeddings disponibles (nodos anteriores a Sprint M-4 o con `generateEmbedding()` retornando `null`), `hybridScore()` hace fallback a `node.weight` puro, garantizando compatibilidad hacia atrás. La función `cosineSimilarity()` verifica dimensiones iguales y retorna 0 si algún vector es nulo.

6.3 NexoContext — Estructura del Resultado

Campo	Tipo	Función en el System Prompt
topNodes	NexoContextNode[]	Hasta 5 nodos inyectados — title, category, summary, weight, daysAgo, url
totalNodes	number	Total de nodos activos del usuario (KPI de memoria)

Campo	Tipo	Función en el System Prompt
clusters	string[]	Categorías dominantes ordenadas por frecuencia
nodeIds	string[]	IDs para que el Attention Engine sepa cuáles reforzar
proactiveNodes	NexoContextNode[]	Nodos con score ≥ 0.65 que SOFIAA menciona proactivamente
topScore	number	Score más alto — útil para analytics y debug del retrieval

6.4 Proactive Surface — Sprint M-5

Los nodos con `hybridScore` $\geq$ `NEXO_PROACTIVE_THRESHOLD` (0.65) activan la superficie proactiva: SOFIAA los menciona explícitamente en su respuesta aunque el usuario no los haya pedido. Ejemplo: si el usuario pregunta '¿dónde comer hoy?' y tiene un nodo de restaurante con score 0.80, SOFIAA puede sugerir directamente ese lugar. El límite de 2 nodos proactivos evita que el sistema se vuelva ruidoso o sobreintrusivo.

7. Attention Engine — Refuerzo por Uso (Sprint M-2)

El Attention Engine implementa el principio de refuerzo por uso: los nodos que SOFIAA usa activamente en conversación se fortalecen, mientras los ignorados continúan decayendo. Este mecanismo crea un ciclo de feedback positivo donde la memoria más útil se vuelve más accesible, análogo al efecto de la repetición espaciada en la memoria humana.

7.1 Mecanismo de Activación

El Attention Engine se activa via `waitUntil()` en el chat route de SOFIAA. Esto significa que corre de forma asíncrona después de enviar la respuesta al usuario, sin añadir latencia al stream de texto. Recibe el array `nodelds` del `NexoContext` (los nodos seleccionados para esa conversación) y aplica refuerzo en batch.

7.2 Lógica de Refuerzo

Campo Actualizado	Fórmula	Límite
<code>weight</code>	<code>min(1.0, weight + ATTEND_DELTA)</code>	<code>MAX_WEIGHT = 1.0</code>
<code>lastReinforced</code>	<code>Date.now()</code>	Resetea el reloj del Decay Engine
<code>reinforceCount</code>	<code>min(9999, reinforceCount + 1)</code>	<code>MAX_REINFORCE_COUNT = 9999</code>

Los nodos de tipo 'insight' (generados por el Reflection Engine) son excluidos del refuerzo por atención — su `decayRate` ya es 0.02 y se quiere que persistan naturalmente sin depender del patrón de uso. El método es silencioso: cualquier error se captura y descarta para nunca interferir con el flujo del chat.

7.3 Panel Mi Grafo — Badge de Nodos Reforzados

La página `/nexo/mi-grafo` muestra el grafo de memoria del usuario con un badge especial () en los nodos que han sido reforzados al menos una vez (`reinforceCount > 0`). Esto da visibilidad al usuario sobre qué partes de su memoria SOFIAA ha utilizado activamente, cerrando el ciclo de transparencia cognitiva.

8. Reflection Engine — Metacognición Sintética (Sprint M-0)

El Reflection Engine es el motor de metacognición de N.E.X.O. No captura conocimiento del mundo exterior, sino que analiza el grafo del propio usuario y genera insights sobre los patrones emergentes en su memoria. Es el sistema que le permite a SOFIAA decir 'noto que últimamente te interesa mucho la fotografía de producto — ¿estás trabajando en algo nuevo?'

8.1 Proceso de Reflexión

runReflection(uid) ejecuta el siguiente flujo:

- 1. Verificar cooldown: si existe un 'insight' con createdAt < 20h → skipped=true
- 2. getNextNodes(uid) → ordenados por weight desc
- 3. Filtrar: solo type='captured' AND weight > 0.1 → top 30
- 4. Construir nodeLines: ' [{category}] {title}: {summary} '
- 5. callGroq(prompt, { maxTokens:1024, temperature:0.75, json:true })
- 6. Parsear JSON → { insights[], patterns[], connections[] }
- 7. Guardar insights como NexoNode type='insight' en Firestore

8.2 Temperatura Alta para Diversidad

El Reflection Engine usa temperature=0.75, significativamente más alto que el Classifier (0.1) o el PDF Engine (0.3). Esta elección es deliberada: la reflexión cognitiva requiere creatividad e identificación de conexiones no obvias, no precisión clasificatoria. Una temperatura baja generaría reflexiones genéricas y predecibles; una temperatura alta produce observaciones sorprendentes y genuinamente útiles.

8.3 Los 4 Tipos de Patrón

Tipo de Patrón	Descripción	Ejemplo
curiosidad	Interés nuevo o exploratorio detectado	'El usuario está explorando herramientas de video por primera vez'
hábito	Comportamiento recurrente en el grafo	'Captura restaurantes todos los viernes — patrón de planeación social'
interés_emergente	Tema que crece en frecuencia en el grafo	'Equipos Sony aumentaron en el grafo — posible proyecto fotográfico'
conexión_temática	Relación no obvia entre dos categorías	'Investigación sobre luz natural y lista de restaurantes con terraza'

8.4 Nodos Insight — Propiedades Especiales

Los insights se guardan como NexoNode con `type='insight'`, `decayRate=0.02` (muy lento), `weight=0.85` e `importanceScore=0.85`. El `nodeId` sigue el formato `'nexo:insight:{timestamp}:{index}'`. Son excluidos del Attention Engine (no se refuerzan por uso) y del propio Reflection Engine (no se analizan a sí mismos — solo nodos capturados). El CRON de reflexión corre a las 10 AM UTC (4 AM CST), una hora después del Decay Engine.

8.5 Umbral Mínimo de Datos

`runReflection()` requiere al menos 3 nodos capturados con peso > 0.1 para generar una reflexión. Si hay menos, retorna `skipped=true` con `reason='Pocos nodos para reflexionar (N)'`. Este umbral evita reflexiones vacías o triviales cuando el grafo del usuario está recién poblado. El CRON multi-usuario aplica esto a todos los usuarios vía `collectionGroup query`, procesando cada UID encontrado.

9. PDF Library — Biblioteca Personal (Sprint M-1)

La Biblioteca Personal permite a los usuarios subir documentos PDF que se procesan automáticamente en múltiples NexoNodes. A diferencia de las capturas web (un nodo por URL), un PDF puede contener múltiples temas, capítulos y conceptos que se convierten en nodos independientes — conectados por el campo `entities.extra.docId` que permite regresar al documento origen.

9.1 Pipeline de Procesamiento PDF

```
POST /api/nexo/pdf recibe: FormData { file: File (application/pdf) }
Límite: 10 MB en Vercel Pro, 4 MB en Hobby

1. Guardar metadata inicial: status='processing' en users/{uid}/biblioteca/{docId}
2. pdf-parse(buffer) → texto plano del PDF (truncado a 8,000 chars para TPM)
3. callGroq(prompt, { maxTokens:2048, temperature:0.3, json:true })
   → { title, author, language, nodes[]: { title, summary, category, entities } }
4. Crear NexoNode por cada topic (máx 7 por documento)
5. Actualizar BibliotecaDoc: status='processed', nodesCreated, title, author
```

9.2 Propiedades Especiales de Nodos PDF

Propiedad	Valor	Justificación
source	'pdf_library'	Distingue de capturas web para queries específicas
weight	0.75	Mayor que capturas web (0.60) — fue subido intencionalmente
decayRate	0.02	Libros decaen como insights — son conocimiento intencional
entities.extra.docId	ID del BibliotecaDoc	Permite regresar al documento origen
entities.extra.source_title	Título del documento	Contexto para el LLM al referenciar el nodo
entities.extra.author	Autor del documento	Provenance del conocimiento

9.3 Eliminación Cascada

`DELETE /api/nexo/pdf?docId={id}` elimina el documento y todos sus nodos asociados en un batch atómico. La query busca nodos con `source='pdf_library'` cuyo `entities.extra.docId` coincida. Esto garantiza que eliminar un documento de la biblioteca también limpia el grafo de los nodos que generó, manteniendo consistencia referencial.

10. Integración Multi-Motor — Publicadores del Ecosistema SOFIAA

N.E.X.O. no es solo un sistema de capturas web personales. Es el grafo cognitivo compartido de todo el ecosistema SOFIAA. Tres extensiones publican sus entidades al grafo mediante publishers especializados, convirtiendo a N.E.X.O. en la capa semántica unificada del sistema.

Motor Publicador	Sprint	Nodos Generados	Source
TEC Bii Cognitive Publisher	T2-1	1 nodo por entidad (proyecto, brief, empleado...)	'tec_bii'
PROMETEO Brand DNA Publisher	P-1	6 nodos por cliente (arquetipo, valores, tono, tabús, promesas, voz)	'prometeo'
Marketing Sofia Publisher	P-8	Nodos por cliente SMM + métricas de performance	'marketing_sofia'

10.1 Flujo Común de Publicación

Los tres publishers siguen el mismo patrón arquitectónico, definido inicialmente en cognitive-publisher.ts de TEC Bii y replicado en los módulos siguientes:

```
1. buildSummaryPrompt(entity) → prompt específico por tipo de entidad
2. callGroq(prompt, { maxTokens:180, temperature:0.3 }) → aiSummary
3. generateEmbedding(title + '. ' + aiSummary) → vector
4. upsertNexoNode(uid, node) → inserta o actualiza en nexo_nodes
5. docRef.update({ nexoNodeId, aiSummary, lastCognitiveSync }) → marca entidad
```

10.2 Diferencias por Publisher

Publisher	decayRate	importanceScore	nodeId Pattern
TEC Bii Cognitive Publisher	0.03	Calculado (calcularImportancia)	'tec-bii:{type}:{entityId}'
PROMETEO Brand DNA Publisher	NEXO_DECAY_RATE (0.05)	0.85 (fijo — alta importancia)	'nexo:brand_{nodeType}:{slug}'
Marketing Sofia Publisher	NEXO_DECAY_RATE (0.05)	Variable por tipo de nodo	'nexo:mm_{type}:{slug}'

10.3 Cross-Domain Reasoning en TEC Bii

El Cross-Domain Engine de TEC Bii (Sprint T2-4) aprovecha la coexistencia de nodos TEC Bii y capturas web personales en el mismo grafo para generar hipótesis semánticas. Separa los nodos por source ('tec_bii' vs

otros), calcula similitud coseno ≥ 0.60 entre pares, y genera hipótesis en lenguaje natural cuando detecta conexión. El grafo unificado de N.E.X.O. hace posible este razonamiento cruzado sin infraestructura adicional.

11. Persistencia y Operaciones Firestore

N.E.X.O. usa Firebase Firestore (Admin SDK, server-side) como su única capa de persistencia. El schema de colecciones está diseñado para escalar horizontalmente por usuario y soportar las queries de los motores de decay, retrieval y reflection sin índices compuestos complejos.

11.1 Paths de Colecciones

Path	Contenido	Queries Principales
users/{uid}/nexo_nodes/{nodeId}	NexoNode completo con embedding	where('weight','>',0.05) + orderBy, collectionGroup
users/{uid}/nexo_events/{eventId}	NexoEvent de pipeline (audit)	orderBy('timestamp') — panel N.O.R.A.
users/{uid}/biblioteca/{docId}	BibliotecaDoc (metadata de PDFs)	orderBy('createdAt') — panel biblioteca

11.2 upsertNexoNode() — Deduplicación Inteligente

La función `upsertNexoNode()` implementa una estrategia de merge inteligente: antes de crear un nuevo nodo, verifica si ya existe uno con la misma URL (`col.where('url','==',node.url).limit(1).get()`). Si existe, aplica un boost de +0.15 al peso y actualiza `lastReinforced`, sin crear duplicado. Si no existe, crea el nodo con `doc().set(node)`. Los nodos sin URL (screenshots, TEC Bii, PROMETEO) siempre crean documento nuevo, usando el `nodeId` como clave de idempotencia.

11.3 Operaciones de Privacidad

N.E.X.O. expone dos operaciones de control de privacidad sobre el grafo: `deleteNexoNode(uid, nodeId)` para eliminar un nodo individual, y `clearAllNexoNodes(uid)` para borrar todo el grafo del usuario. Esta última usa un batch delete que procesa todos los documentos de la colección en una sola operación atómica. Ambas operaciones están disponibles en el panel `/nexo/mi-grafo` donde el usuario puede ver y controlar su memoria.

12. Chrome Extension MVP — Canal de Captura Activa (Sprint N-1)

La Chrome Extension es el canal principal de ingesta de contenido web. Permite al usuario capturar el contenido de cualquier página con un solo clic, enviando el título, URL y texto DOM extraído del `page.innerText` al endpoint `/api/nexo/ingest` con el ID token de Firebase para autenticación.

12.1 Arquitectura de la Extensión

La extensión sigue el patrón manifest v3 con service worker. Tiene tres componentes: `popup.html` (interfaz de captura con botón y feedback visual), `content.js` (extrae `page.innerText` desde el tab activo), y `background.js` (envía el request autenticado a `/api/nexo/ingest`). La autenticación Firebase fluye desde la sesión web de SOFIAA hacia la extensión via `chrome.storage.local` donde se persiste el ID token.

12.2 PWA Share Target — Captura Mobile (Sprint N-6)

En dispositivos móviles, N.E.X.O. opera como PWA Share Target registrada en el `manifest.json` de SOFIAA. Cuando el usuario comparte cualquier URL o texto desde otra app (Safari, Instagram, Maps), iOS/Android abre SOFIAA como destino del share con el contenido pre-llenado. El flujo es idéntico al de la extensión: autenticación Firebase + POST `/api/nexo/ingest` con `source='pwa_share'`.

13. Privacy Dashboard — Mi Grafo (Sprint N-7)

El panel /nexo/mi-grafo es la interfaz de transparencia cognitiva del sistema. Permite al usuario visualizar su grafo de memoria, ver qué nodos tiene SOFIAA sobre él, identificar cuáles ha usado activamente (badge ) y eliminar los que no quiere que persistan. Este panel fue diseñado desde el Sprint N-7 como requisito ético fundamental: el usuario tiene control total sobre su propia memoria.

Elemento UI	Dato Mostrado	Fuente
Contador total	Total de nodos en el grafo	getNexoNodes().length
Nodo card	Title, category, summary, daysAgo, weight	NexoNode fields
Badge 	reinforceCount > 0 → nodo usado por SOFIAA	reinforceCount field (Sprint M-2)
Badge de peso	0.0–1.0 → porcentaje de relevancia actual	weight field
Botón eliminar	deleteNexoNode(uid, nodeId)	DELETE /api/nexo/{nodeId}
Botón vaciar	clearAllNexoNodes(uid)	DELETE /api/nexo?clear=all
Sección Biblioteca	Lista de PDFs procesados	users/{uid}/biblioteca

14. Historia de Sprints — De Captura a Cognición

N.E.X.O. evolucionó en dos fases: los sprints N-0 a N-7 construyeron la infraestructura de captura y persistencia; los sprints M-0 a M-4B añadieron inteligencia cognitiva progresiva. El sistema pasó de ser un 'bookmark inteligente' a un grafo de experiencia que aprende, refuerza, decae y reflexiona.

Sprint	Nombre	Capacidad Añadida
N-0	Contratos y Fundación	NexoNode schema, NexoCategory, constantes, Firestore helpers
N-1	Chrome Extension MVP	Extensión manifest v3, captura desde browser con Firebase auth
N-2	Pipeline de Ingesta LLM	POST /api/nexo/ingest, LLM Classifier con Groq, NexoEvent
N-3	Decay Engine	GET /api/nexo/decay, applyNexoDecay(), lambda adaptativo por categoría
N-4	Context Injection	getNexoContext(), inyección en system prompt, top-5 por peso
N-5	Action Cards	Tarjetas de acción en el chat basadas en nodos NEXO
N-6	PWA Share Target Mobile	manifest.json share_target, captura desde móvil
N-7	Privacy Dashboard	/nexo/mi-grafo — visualización y control del grafo
M-0	Reflection Engine	runReflection(), nodos tipo 'insight', CRON 4 AM CST
M-1	PDF Library	processPdf(), BibliotecaDoc, /nexo/biblioteca, pdf-parse + Groq
M-2	Attention Engine	attendNexoNodes(), reinforceCount, waitUntil(), badge 
M-3	Cognitive Profile	CognitiveProfile schema, detección rule-based de señales cognitivas
M-4	Semantic Retrieval	embedding en NexoNode, hybridScore(), getSemanticNexoContext()
M-4B	Backfill Embeddings	script backfill-embeddings.mjs para nodos pre-M-4

15. Diseño Cognitivo — Principios Teóricos

N.E.X.O. está fundamentado en principios de neurociencia cognitiva y teoría de memoria. No es una implementación ad-hoc, sino una transposición deliberada de mecanismos cognitivos conocidos a un sistema computacional.

15.1 Curva de Ebbinghaus — Olvido Adaptativo

El Decay Engine implementa la curva de olvido de Ebbinghaus: sin repetición, el conocimiento decae exponencialmente. La innovación de N.E.X.O. es que la 'repetición' no es explícita (el usuario no hace flashcards) sino implícita: cada vez que SOFIAA referencia un nodo en conversación, el Attention Engine reinicia el reloj del decay, simulando una repetición espaciada automática y pasiva.

15.2 Consolidación Selectiva — Importancia vs. Frecuencia

El sistema distingue entre importancia (importanceScore, asignada en ingesta por el LLM) y frecuencia de uso (reinforceCount, acumulada por el Attention Engine). Ambas dimensiones son independientes: un nodo importante pero nunca usado decae si no es reforzado; un nodo frecuentemente usado pero poco importante puede decaer más lento gracias al Attention Engine. El hybridScore combina estas dos dimensiones con el peso temporal actual.

15.3 Metacognición — El Reflection Engine

La reflexión cognitiva es el mecanismo de metacognición del sistema: analizar los propios patrones de pensamiento. El Reflection Engine no captura nuevo conocimiento del mundo, sino que genera insights sobre el propio grafo del usuario, creando un segundo orden de conocimiento (conocimiento sobre el conocimiento). Los insights se almacenan como nodos 'insight' con mayor peso y menor decaimiento, reflejando que el conocimiento sintetizado es más valioso que la materia prima.

15.4 Memoria Episódica y Semántica

El sistema distingue implícitamente entre memoria episódica (capturas con URL, tiempo y contexto específico) y memoria semántica (insights y Brand DNA sin URL específica). Las capturas web tienen URL y capturedAt como contexto episódico; los insights y nodos PROMETEO/TEC Bii son semánticos: su valor está en el concepto, no en cuándo o dónde se capturaron.

16. Modelo de Datos Completo — Paths Firestore

Path Firestore	Tipo de Dato	Acceso Principal
users/{uid}/nexo_nodes/{nodeId}	NexoNode (schema completo)	Server-side: getAdminDb()
users/{uid}/nexo_events/{eventId}	NexoEvent (audit trail)	Server-side: logNexoEvent()
users/{uid}/biblioteca/{docId}	BibliotecaDoc (metadata PDF)	Server-side: processPdf()
users/{uid}/tec_bii_*/nexoNodeId	string (ref al grafo)	Vínculo entidad→NexoNode
smm_workspaces/{wsId}/prometeo_*/nexoNodeId	string (ref al grafo)	Vínculo Brand DNA→NexoNode

El diseño multi-tenant por UID garantiza aislamiento completo entre usuarios. `collectionGroup('nexo_nodes')` se usa en los CRONs de decay y reflection para descubrir automáticamente todos los UIDs con datos activos sin mantener un índice de usuarios separado. La colección `nexo_events` es el audit trail del sistema: registra cada captura, refuerzo, decay y eliminación para el panel de telemetría N.O.R.A.

17. API Endpoints de N.E.X.O.

Endpoint	Método	Autenticación	Función
POST /api/nexo/ingest	POST	Firebase ID token	Clasifica + persiste NexoNode desde captura web/screenshot/manual
GET /api/nexo/decay	GET	CRON_SECRET Bearer	Aplica decay exponencial a todos los usuarios (CRON 3 AM CST)
GET /api/nexo/reflect	GET	CRON_SECRET Bearer	Reflection Engine para todos los usuarios (CRON 4 AM CST)
POST /api/nexo/reflect	POST	CRON_SECRET Bearer	Reflection manual para uid específico { uid }
POST /api/nexo/pdf	POST	Firebase ID token	Procesa PDF → múltiples NexoNodes via Groq
DELETE /api/nexo/pdf	DELETE	Firebase ID token	Elimina BibliotecaDoc + todos sus NexoNodes asociados

18. Conclusión — La Memoria como Infraestructura Cognitiva

N.E.X.O. representa un paradigma diferente de 'memoria de IA': en lugar de depender de un historial de conversación o de bases de datos consultadas por keywords, construye un grafo de experiencia que evoluciona con el usuario, aprende de sus patrones y decae donde ya no es relevante. Es la primera capa del stack cognitivo de SOFIAA — sin ella, todas las extensiones (PROMETEO, TEC Bii, ATENA, ALEJANDRÍA) estarían desconectadas del contexto real del usuario.

La arquitectura de siete motores crea un ciclo completo de vida del conocimiento: Ingest captura → Classifier estructura → Persistence almacena → Context Injection conecta con el chat → Attention Engine refuerza → Decay Engine olvida → Reflection Engine sintetiza. Este ciclo es autónomo, opera 24/7 via CRONs, y es transparente: el usuario puede ver, controlar y eliminar cualquier nodo desde el panel Mi Grafo.

El diseño futuro contempla la activación completa del Semantic Retrieval Engine con embeddings reales cuando se integre un proveedor de embeddings compatible (actualmente generateEmbedding() retorna null con fallback a peso puro). La infraestructura de hybridScore() está lista — solo espera el proveedor vectorial. Esto convertiría N.E.X.O. de un sistema de retrieval por peso a un sistema de retrieval semántico completo, cerrando el ciclo de la visión original de Sprint M-4.

Capacidad	Estado	Descripción
Ingesta web (Chrome/PWA)	✓ Activo	Captura multi-canal con clasificación LLM
Decay Engine diario	✓ Activo	Lambda adaptativo por categoría · CRON 3 AM CST
Context Injection (top-5)	✓ Activo	Inyección en system prompt de SOFIAA
Reflection Engine (insights)	✓ Activo	CRON 4 AM CST · cooldown 20h
PDF Library	✓ Activo	pdf-parse + Groq · máx 7 nodos/doc
Attention Engine	✓ Activo	waitUntil() · ATTEND_DELTA=0.03
Proactive Surface	✓ Activo	score ≥ 0.65 · máx 2 nodos
Semantic Retrieval (embeddings)	⌚ Parcial	hybridScore() listo · generateEmbedding() retorna null
TEC Bii / PROMETEO / SMM publishers	✓ Activo	3 publishers externos publican al grafo N.E.X.O.
Cross-Domain NEXO↔TEC Bii	✓ Activo	Cosine similarity ≥0.60 · hipótesis generadas por Groq

